

Speech AI Models, Explained Plainly

A field guide to every kind of model you will meet while building a Yiddish speech-to-text and text-to-speech system. Written for someone who has never trained a model.

How to read this

Part 1 is the map: what jobs need doing and which model does each. Part 2 explains the handful of building blocks that every model is made of. Parts 3 to 5 walk through the actual named models for hearing speech, converting text, and making speech. Part 6 is a decision table for this project. A glossary is at the end. Skip around freely.

1. The map: what the pieces are and how they connect
2. The building blocks every model is made from
3. Models that hear: speech recognition (ASR)
4. Models that read: text, letters, and pronunciation (G2P)
5. Models that speak: text-to-speech (TTS)
6. Helper models you will bump into
7. Choosing for this project
8. Glossary

1. The map

Everything in this project sits on one of two conveyor belts. One belt turns sound into text. The other turns text into sound. Each belt has stations, and each station is a separate kind of model.

HEARING BELT

audio → [ASR model] → words → [alignment model] → words with timestamps

SPEAKING BELT

words → [G2P] → sounds (IPA) → [TTS acoustic model] → [vocoder] → audio

Here is what each station does, in one sentence each.

Station	Job	In this project
ASR (automatic speech recognition)	Listens to audio and writes down the words.	Yiddish Labs API produced the transcripts. Parakeet is the candidate to fine-tune yourself.
Alignment	Given audio and its text, says exactly when each word was said.	Yiddish Labs gave phrase-level timestamps, which the segmentation script used to cut clips.

G2P (grapheme-to-phoneme)	Turns spelled words into the sounds actually pronounced.	The Phonikud-yi engine plus the lexicon built from the phonetic index.
TTS acoustic model	Turns a sequence of sounds into a "picture" or code of the speech.	Qwen3-TTS and other candidates.
Vocoder / codec decoder	Turns that picture or code into an actual waveform you can play.	Bundled inside modern TTS models.

Why Yiddish needs the G2P station at all: English TTS systems mostly learn spelling-to-sound on their own because there is a mountain of English data. Yiddish has a small mountain, and worse, Hebrew-origin words keep their ancient spelling while the pronunciation drifted centuries ago. שבת is spelled sh-b-t but said *shabes*. No model will guess that from a few hundred hours of audio. So we hand it the answer in the form of IPA.

2. The building blocks

Every model on either belt is assembled from a small set of ideas. Learn these ten and every model name becomes readable.

2.1 What a neural network is

A neural network is a very large formula with millions of adjustable knobs, called parameters or weights. Training means showing it examples (audio paired with text, say), measuring how wrong its answer is, and nudging every knob a tiny bit in the direction that would have made it less wrong. Repeat millions of times. "Fine-tuning" means starting from a network someone already trained and nudging it a little more on your own examples. It is far cheaper than starting from zero and it is what you will do for Yiddish.

2.2 The three classic shapes

CNN (convolutional network)

Slides a small window across the input and looks for local patterns. Great for "what is happening in this 25 millisecond slice of audio" or "what is in this corner of an image." Almost every audio model has a CNN at the very front to turn raw sound into a compact stream of features.

RNN / LSTM (recurrent network)

Reads the input one step at a time, carrying a memory forward. Natural for sequences, but slow to train because step 500 cannot start until step 499 is done, and the memory fades over long inputs. Mostly replaced by Transformers, but still inside some ASR models (the "T" in RNN-T).

Transformer

The architecture behind GPT, Whisper, and nearly everything since 2018. Its trick is **attention**: every position in the input can look directly at every other position and decide what matters. No fading memory, and every step can be computed at once, so training is fast on GPUs. The cost is that attention grows with the square of the input length, which is why audio models compress sound into a short stream of features first.

2.3 Encoder, decoder, and both

An **encoder** reads the whole input and produces a rich internal summary. A **decoder** writes the output one piece at a time, looking back at what it has already written (and at the encoder's summary, if there is one).

- **Encoder-only:** BERT-style. Reads text or audio and labels it. Phonikud is an encoder that reads unpointed Hebrew letters and predicts the vowel marks under each one.
- **Decoder-only:** GPT-style. Generates a sequence left to right. Modern "LLM-style" TTS models are decoder-only networks that generate audio tokens instead of word tokens.
- **Encoder-decoder:** Whisper. An encoder listens to 30 seconds of audio; a decoder writes the transcript word by word while attending to the encoder's summary.

2.4 Autoregressive versus parallel

Autoregressive means generating one piece at a time, each piece conditioned on the previous ones. It is how GPT writes and how Whisper transcribes. Quality is high because each step sees full context, but generation is slow and can "run away" (repeat a phrase forever, or skip a sentence).

Non-autoregressive or parallel models produce the whole output in one shot or in a fixed number of refinement passes. Faster and more stable; historically slightly less natural. Diffusion and flow matching (below) are how parallel models got their quality back.

2.5 How ASR turns a long audio into a short text: CTC and Transducers

Audio has, say, 1,000 frames; the sentence has 12 words. Something must decide which frames belong to which word. Two standard answers:

CTC (connectionist temporal classification)

Every frame emits one letter or a special "blank." Repeated letters and blanks are collapsed afterwards: *hh-e-blank-l-l-blank-o* becomes *hello*. Simple, fast, needs no decoder. Each frame guesses independently, so it cannot use context well; a language model is often bolted on to fix spelling.

Transducer (RNN-T, or TDT in Parakeet)

Like CTC but with a small predictor network that remembers what was just emitted, so it can use context. Streams in real time, which is why phones and NVIDIA's Parakeet use it. TDT ("token and duration transducer") also predicts how many frames to skip, which makes it very fast.

Attention encoder-decoder (Whisper)

No frame-by-frame alignment at all. The decoder just writes the text while attention figures out where to look. Most accurate with context and punctuation, but needs the whole chunk before it starts and can hallucinate on silence.

2.6 Generative building blocks: how models make new audio

Recognition models classify. Speech models must *create*. There are five families of creative mechanism, and TTS model names usually tell you which one is inside.

VAE (variational autoencoder)

Squeezes audio into a small code and learns to expand it back. Gives you a compact "latent" space to generate in. VITS is built around one.

GAN (generative adversarial network)

Two networks: a generator makes audio, a critic tries to tell it from real audio, and they train against each other. Produces crisp, fast output. HiFi-GAN and most vocoders are GANs.

Normalizing flow

A chain of reversible math steps that turns simple noise into a complex distribution and back. Exact and fast, used inside VITS and Glow-TTS.

Diffusion

Training: take real audio (or its spectrogram), add noise step by step until it is static, and teach the network to undo one step. Generation: start from pure static and denoise repeatedly until speech

appears. Extremely high quality; the original versions needed 50 to 1,000 passes and were slow. This is the same idea behind image generators like Stable Diffusion.

Flow matching

Diffusion's faster cousin. Instead of learning to undo noise step by step, the network learns a straight-line path from noise to data, so 8 to 32 passes give the same quality diffusion needed hundreds for. F5-TTS, Voicebox, and the second stage of CosyVoice use it. When a 2024 or later paper says "diffusion," it usually means flow matching under the hood.

2.7 Neural audio codecs and audio tokens

A codec compresses audio into a short list of integers (tokens), typically 25 to 75 per second, and can decode them back into sound. Once audio is integers, a text-style language model can predict it exactly the way GPT predicts words. This single idea created the "LLM-style TTS" family: EnCodec, SNAC, WavTokenizer, and Qwen's own tokenizer are codecs; VALL-E, XTTS, Fish Speech, CosyVoice and Qwen3-TTS are language models that write codec tokens.

2.8 Self-supervised pretraining

Labelled audio is scarce, unlabelled audio is endless. Self-supervised models (wav2vec 2.0, HuBERT, WavLM, XEUS) listen to hundreds of thousands of hours of raw audio with no transcripts, learning to predict masked-out chunks. The result is an encoder that already "understands" speech and needs only a little labelled data to become an ASR system. This is why fine-tuning works with tens of hours instead of thousands.

2.9 Spectrograms and mel

Raw audio is 16,000 to 48,000 numbers per second, too many to reason about directly. A spectrogram slices audio into 10 to 25 millisecond frames and records how much energy is at each pitch, giving a picture where time runs left to right and pitch bottom to top. "Mel" means the pitch axis is warped to match human hearing. Most models work on mel spectrograms and a vocoder turns the spectrogram back into sound.

2.10 Tokenizers

Text models do not read letters; they read tokens, chunks of characters chosen by frequency. A tokenizer built on English text has no idea what to do with Hebrew letters and will shatter Yiddish into single characters or unknowns. This is why Parakeet v3 needs a new SentencePiece tokenizer trained on your transcripts before fine-tuning, and why Whisper and NVIDIA's newer Nemotron 3.5 ASR, whose tokenizers already include Hebrew script, are easier to start with.

3. Models that hear: ASR

Model	Architecture	What it is good and bad at	For Yiddish
Whisper (OpenAI)	Transformer encoder-decoder, autoregressive. Sizes from 39M to 1.5B parameters. Current best: large-v3 and the 5x faster large-v3-turbo. MIT licence.	Trained on 680k hours across 99 languages including Yiddish (badly) and Hebrew. Produces punctuation and handles noise well. Slow, works in 30-second windows, and hallucinates on silence or music.	Easiest to fine-tune; tokenizer already knows Hebrew script. Yiddish Labs' service is very likely a fine-tuned Whisper-class model.
yi-whisper-large-v3 (ivrit.ai)	Whisper large-v3 fine-tuned for Yiddish, October 2025. Apache-2.0.	Trained on about 97 hours of crowd-sourced Yiddish: 78 h of read Wikipedia-style articles and 19 h of WhatsApp voice notes. No published error rate yet. Language token must be forced to Yiddish; translation is broken.	The starting point. Already speaks Yiddish; fine-tune it on your 10 h of Hasidic podcast speech rather than starting from stock Whisper. Their two training datasets are public and add 97 h to yours.
Parakeet TDT 0.6B v3 (NVIDIA, Aug 2025)	FastConformer encoder (CNN plus Transformer) with a TDT transducer head. 0.6B parameters, CC-BY-4.0.	Trained on 670k hours; beats Whisper large-v3 on 25 European languages at a quarter of the size and runs fast on a CPU. Tokenizer is 8,192 SentencePiece pieces built from those 25 languages only.	No Hebrew script in the tokenizer, so a new tokenizer and a rebuilt decoder are needed before fine-tuning. Works, but see the next row.
Nemotron 3.5 ASR streaming 0.6B (NVIDIA, June 2026)	Same FastConformer-transducer family as Parakeet, one checkpoint for 40 locales, latency adjustable from 80 ms to 1 s.	NVIDIA's newest open ASR. Hebrew is in the tokenizer but listed as "not production tuned"; NVIDIA publishes a fine-tuning guide for exactly that case.	The better NeMo target for Yiddish: Hebrew letters already tokenize, so your Parakeet manifests work unchanged and no tokenizer rebuild is needed.
Conformer / FastConformer	The encoder inside Parakeet: convolution blocks interleaved with attention.	The standard ASR encoder since 2020. Convolution catches local sound patterns, attention catches long context.	What you are fine-tuning when you fine-tune Parakeet.

wav2vec 2.0 / XLS-R / MMS (Meta)	Self-supervised Transformer encoder, usually with a CTC head.	MMS covers over 1,000 languages. Fine-tunes with very little labelled audio. CTC output has no punctuation and needs a language model for spelling.	Best choice if you had only 1 to 3 hours. With 10+ hours Whisper or Parakeet will beat it.
HuBERT / WavLM	Self-supervised encoders like wav2vec but trained by predicting cluster labels.	Better speaker and prosody information than wav2vec. Often used as the "understanding" front end inside TTS and voice conversion.	Not a transcription model on its own; appears inside other tools.
XEUS / PhoneticXeus	Self-supervised encoder trained on 4,000+ languages with a phone (sound) recogniser head.	Outputs IPA-like phones rather than words, regardless of language.	Phonikud-yi uses it as audio evidence for how words are actually pronounced.
Canary 1B v2, SeamlessM4T, Qwen-Audio	Multilingual encoder-decoder or audio-LLM models.	Transcribe and translate in one model, with word timestamps in Canary's case. Larger and slower than the transducers.	None cover Yiddish out of the box; Canary v2 is the same 25 European languages as Parakeet v3.

Plain-English summary

Every modern ASR model is a listener (encoder) plus a writer. The writers differ: CTC writes one letter per frame and collapses repeats; a transducer does the same with a bit of memory; Whisper's decoder writes freely like GPT. Whisper is the most forgiving to fine-tune, Parakeet the fastest to run, wav2vec-family the hungriest for little data.

4. Models that read: text, letters, pronunciation

4.1 G2P: from spelling to sounds

Four ways to get from a written word to its pronunciation, from dumbest to smartest. Real systems, including Phonikud-yi, chain them.

Lexicon lookup

A dictionary: word in, IPA out. Perfect for words it knows, useless for words it does not. The lexicon you built is this. Every serious TTS keeps one in front of everything else because it is the only component a human can correct with certainty.

Rules

Hand-written letter-to-sound rules. Works for the Germanic three quarters of Yiddish, which is spelled phonetically. Fails on Hebrew-origin words, which is why they need the lexicon.

Sequence-to-sequence G2P

A small Transformer or LSTM that reads letters and writes phonemes, trained on lexicon pairs. Handles unseen words by analogy. Needs several thousand verified pairs to beat rules; below that it mostly memorises the rules' mistakes.

Diacritisation model (Phonikud, Nakdimon, Dicta)

An encoder-only character model (a BERT for letters) that predicts the vowel marks under each Hebrew letter in context. Context matters because the same spelling can be two words. Phonikud-yi is a 306M-parameter version trained for Hasidic Yiddish. Once the vowels are known, rules finish the job.

LLM as G2P

Ask a large language model to transcribe the word. Works surprisingly often, inconsistently, and costs money per call. Useful for building candidate lists to hand to a native reviewer, not for production.

4.2 Text normalisation

Before any of that, digits, abbreviations, and dates must become words: "9,000" to *nayn toyzent*, an acronym with a gershayim to its spoken form. Usually rules; sometimes a small seq2seq model. The dataset prep script currently drops clips containing digits precisely because this step does not exist yet for Yiddish.

5. Models that speak: TTS

TTS models come in four generations. Each is still in use, and the names on Hugging Face tell you which generation you are looking at.

5.1 Generation one: two-stage, spectrogram plus vocoder

Tacotron 2 (2017) reads phonemes with an encoder and writes a mel spectrogram autoregressively; **WaveNet** or later **HiFi-GAN** turns the spectrogram into sound. **FastSpeech 2** (2020) does the same in parallel using a duration predictor, so it never runs away. **Piper** and many "lightweight" voices are this design. Simple, fast, controllable, and sounds slightly robotic. Needs clean single-speaker data and works well with as little as two to five hours.

5.2 Generation two: end-to-end, VITS

VITS (2021) fused the acoustic model, a VAE, a normalizing flow, and a GAN vocoder into one network trained together, and it sounds much more natural than generation one. It is the most fine-tuned open TTS architecture in existence; Coqui, Piper's larger voices, and dozens of language-specific voices are VITS. Trains on a single GPU in a day with a few hours of audio. **Kokoro** (v1.0, January 2025) is a StyleTTS-2-derived tiny model in this spirit, 82M parameters, Apache-2.0, that runs on a CPU and sounds far better than its size suggests. It cannot clone voices and its training code is not public.

5.3 Generation three: LLM-style codec TTS

Turn audio into codec tokens (section 2.7) and train a GPT-style decoder to write them. **VALL-E** (2023) started it; **XTTS v2**, **Fish Speech**, **CosyVoice 2**, **Orpheus**, **Chatterbox**, and **Qwen3-TTS** are the practical ones. Because the core is a language model, you get zero-shot voice cloning from a few seconds of reference audio, natural prosody, and the ability to fine-tune on a new language the same way you would fine-tune a chat model. The costs are speed, occasional runaway generation, and needing hundreds of hours for a genuinely new language rather than a new voice. Most start from a text LLM (Qwen3-TTS from Qwen3, Orpheus from Llama), so they already "know" Hebrew script, which helps.

5.4 Generation four: diffusion and flow-matching TTS

Voicebox (Meta, 2023), **E2-TTS**, and **F5-TTS** (2024) drop the autoregressive decoder entirely. Text and a noisy mel spectrogram go into a Transformer that, in a few flow-matching passes, cleans the noise into speech. No duration model, no runaway, very fast, and excellent cloning. **StyleTTS 2** mixes a diffusion style-predictor with a GAN generator. **CosyVoice** and **Qwen3-TTS** actually use a flow-matching stage after their language model to turn coarse tokens into fine mel detail, so the two families are merging. Flow-matching models need alignment-free training data (just audio plus text) and tolerate noisier data than generation one.

Model	Generation	Strengths	Data it wants
-------	------------	-----------	---------------

Piper	1 / 2 (VITS-lite)	Runs on a Raspberry Pi. Clear, intelligible.	2 to 10 h clean single speaker, phonemes in.
VITS	2	Natural, well documented, trains in a day.	3 to 20 h, one or a few speakers, phonemes in.
Kokoro / StyleTTS 2	2 / 4 mix	Tiny and high quality. Kokoro's training code is not public.	StyleTTS 2 fine-tunes on a few hours.
XTTS v2	3	Cloning from 6 s, 17 languages, easy fine-tune scripts.	New voice: minutes. New language: 100 h+.
Fish Speech 1.5 / CosyVoice 2	3 (+4 stage)	Strong multilingual cloning, open training code. CosyVoice 2 (0.5B) has the best speaker similarity of the small models.	Similar to XTTS.
Fish Audio S2 Pro (2026)	3 (+4 stage)	Open weights, trained on over 10 million hours. Tops the 2026 blind-listening leaderboards, ahead of ElevenLabs. Cross-lingual cloning.	Inference is easy; fine-tuning a big model like this needs serious GPU. Try zero-shot cloning on Yiddish first.
Qwen3-TTS (Jan 2026)	3 (+4 stage)	0.6B and 1.7B, Apache-2.0, built on the Qwen3 LLM so Hebrew script is not foreign to it. Ten official languages, streaming, cloning from 3 s, style by text instruction.	Official recipe is single-speaker only for now (multi-speaker is on their roadmap; community forks add it). JSONL with <code>audio</code> , <code>text</code> , <code>ref_audio</code> ; the manifests now carry that field. Plan on 20 to 50 h for a new language.
Chatterbox / Chatterbox-Turbo (Resemble)	3	MIT licence, clones from 5 s, preferred over ElevenLabs in 2026 blind tests. English-centred.	Multilingual fine-tune scripts exist; Yiddish would be a from-scratch language.
IndexTTS-2	3 (+4 stage)	Separate control of voice and emotion; simplest training loop of the LLM-style models; fast.	Similar to CosyVoice 2.
VoxCPM2 (2026)	3 / 4 hybrid	2B, Apache-2.0, 30 languages, 48 kHz output, runs on CPU.	Worth a zero-shot test; no Yiddish in its 30 languages.
F5-TTS / E2	4	Fast, stable, good cloning, simplest training loop. Papers in 2025 and 2026 show it picking up unseen languages from a few hours plus synthetic data.	Audio plus text only, 20 to 50 h for a new language; tolerant of imperfect labels. Most documented path for a low-resource language.

5.5 Vocoders and codec decoders

The last step. **WaveNet** (2016, autoregressive, slow), **WaveGlow** (flow), **HiFi-GAN** (2020, GAN, the workhorse), **BigVGAN** (2022, better for singing and unseen voices), **Vocos** (2023, fast). Codec decoders (EnCodec, DAC, SNAC) do the same job for token-based models. You almost never train these yourself; they are language-agnostic because they only turn pictures of sound into sound.

Plain-English summary

Generation one draws a picture of the speech then paints sound from it. Generation two does both at once. Generation three treats sound as a foreign language and lets a GPT write it. Generation four starts from static and sculpts it into speech in a few passes. Older generations need less data and phonemes; newer ones need more data but forgive messy labels and can clone voices.

6. Helper models you will meet

VAD (voice activity detection)

Says where speech is and is not. Silero VAD, WebRTC VAD. Used to cut long recordings and drop silence.

Speaker embedding / verification

Turns a voice into a vector; same speaker gives nearby vectors. ECAPA-TDNN, WavLM-SV, Resemblyzer. Used to check that all clips in a "speaker" folder are actually one person and to condition TTS on a voice.

Diarisation

Who spoke when. pyannote. Needed for interview shows before they can be used for single-speaker TTS.

Forced alignment

Given audio and its exact text, returns per-word or per-phone timestamps. Montreal Forced Aligner, wav2vec2 CTC alignment, WhisperX. Would give you much tighter clip boundaries than the phrase-level timestamps used now.

Enhancement / denoising / separation

Removes background noise or music. Demucs, DeepFilterNet, resemble-enhance. Worth running on the shiurim before TTS training; do not run on ASR training data, since ASR should learn real noise.

Voice conversion

Changes who is speaking without changing the words. RVC, kNN-VC, Seed-VC. A shortcut to a target voice when you have a good TTS in a different voice.

Audio LLMs

Models that take audio in and chat about it or answer with speech: GPT-4o audio, Gemini, Qwen2.5-Omni. Not trainable by you at this scale, but usable as a reviewer or labeller.

7. Choosing for this project

Goal	Recommended	Why
Better Yiddish transcription than the paid API	Fine-tune ivrit.ai's yi-whisper-large-v3 first; Nemotron 3.5 ASR second	yi-whisper already speaks Yiddish from 97 h of crowd recordings; your 10 h of Hasidic podcast speech is exactly the domain it lacks. Nemotron 3.5 is the fast, streaming option and its tokenizer already handles Hebrew script, unlike Parakeet v3. Skip stock Whisper and Parakeet v3.

Pronunciation of new words	Keep lexicon plus rules; train a seq2seq G2P only after 5,000 verified words	With 697 verified words a neural G2P would only learn the rules' mistakes. Native review of the 388 most frequent low-confidence words fixes half of all shaky tokens for a day's work.
A clear, dependable Yiddish voice quickly	VITS or Piper on the bulletin reader	Single studio voice, phonemes available, a few hours is enough. Trains overnight on one GPU.
A natural voice with cloning of the rebbes	F5-TTS fine-tune first; Qwen3-TTS 1.7B second	Both take the audio-plus-text data as prepared. F5-TTS trains on all speakers at once and has the most published low-resource results. Qwen3-TTS's official recipe is single-speaker, so train one voice per run, starting with the bulletin reader. Expect to need 20 to 50 h of clean speech and to denoise the shiurim first. Before training anything, run zero-shot cloning with Fish Audio S2 and Qwen3-TTS on a Yiddish sentence; if one is already intelligible, fine-tuning it is the shortest path.
Tighter clip boundaries	Forced alignment with a fine-tuned wav2vec2 or WhisperX	Phrase-level timestamps are within about a quarter second. Word-level alignment removes clipped tails and lets you cut at silences.

The order that gets the most out of the least effort: review the top low-confidence words, fine-tune yi-whisper on your 10 hours plus ivrit.ai's 97, use it to transcribe the remaining 180 hours of yiddish24 for free, align, then train TTS on the enlarged set.

What is newest, checked 3 September 2026

Model	Date	Why it matters here
ivrit.ai yi-whisper-large-v3	Oct 2025	Only public Yiddish ASR. Start from it.
NVIDIA Parakeet TDT 0.6B v3	Aug 2025	25 European languages, no Hebrew script.
NVIDIA Nemotron 3.5 ASR streaming 0.6B	Jun 2026	40 locales including Hebrew; NVIDIA's newest ASR and its fine-tuning guide targets under-tuned languages.
Qwen3-TTS 0.6B / 1.7B	Jan 2026	Open, Apache-2.0, single-speaker fine-tuning recipe published.
Fish Audio S2 Pro	2026	Open weights topping blind-listening tests.
VoxCPM2, Chatterbox-Turbo, IndexTTS-2, CosyVoice 2	2025 to 2026	Strong open cloning models; none list Yiddish.
Whisper large-v3 / turbo	2024	Still OpenAI's latest open weights; nothing newer released.

8. Glossary

Attention

The mechanism that lets each position in a sequence look at every other position and weigh what matters.

Autoregressive

Generating one piece at a time, each conditioned on the previous pieces.

Codec (neural)

A network that compresses audio into a short list of integer tokens and decodes them back.

CTC

A way to train speech recognisers without knowing which frame belongs to which letter: emit a letter or blank per frame, collapse repeats.

Diffusion

Generate by starting from noise and removing it step by step.

Encoder / decoder

The part that reads and summarises, and the part that writes the output.

Fine-tuning

Continuing to train an existing model on your own data.

Flow matching

A faster relative of diffusion that learns a straight path from noise to data.

G2P

Grapheme-to-phoneme: spelling in, sounds out.

GAN

Generator versus critic training; gives crisp fast output; most vocoders.

IPA

International Phonetic Alphabet, one symbol per sound, language-independent.

Latent

A compressed internal representation a model generates in, instead of raw audio.

Mel spectrogram

A picture of sound, time across and pitch up, with pitch warped to human hearing.

Nikud

Hebrew vowel marks. Predicting them is a step toward pronunciation.

Parameters

The adjustable numbers inside a model; "306M" means 306 million of them.

Phoneme

One distinct speech sound.

Self-supervised

Learning from unlabelled audio by predicting hidden parts of it.

Token

The unit a model reads or writes: a chunk of text, or an integer standing for a slice of audio.

Transducer (RNN-T / TDT)

A streaming speech recogniser that emits tokens frame by frame with a small memory of what it already said.

Transformer

The attention-based architecture behind nearly all current models.

VAE

A network that compresses to a small code and reconstructs, giving a latent space to generate in.

Vocoder

Turns a spectrogram into a waveform.

WER

Word error rate, the standard ASR score: fraction of words wrong. Lower is better.

Zero-shot cloning

Imitating a voice from a short sample with no training.

Sources checked on 3 September 2026

ivrit.ai yi-whisper-large-v3 model card, huggingface.co/ivrit-ai/yi-whisper-large-v3 · NVIDIA parakeet-tdt-o.6b-v3 model card, huggingface.co/nvidia/parakeet-tdt-o.6b-v3 · NVIDIA nemotron-3.5-asr-streaming-0.6b model card and the "How to fine-tune Nemotron 3.5 ASR for your language" post on the Hugging Face blog · NVIDIA Nemotron speech FAQ on production-ready European models, perspectives.nvidia.com · Qwen3-TTS repository and finetuning README, github.com/QwenLM/Qwen3-TTS, and the Qwen3-TTS technical report, arXiv 2601.15621 · OpenAI Whisper repository, github.com/openai/whisper · IndexTTS-2 paper, arXiv 2502.05512 · 2026 open TTS round-ups from BentoML, Modal, Resemble AI, SiliconFlow and Pinggy.

Prepared 3 September 2026 for the Kohn AI Yiddish TTS project.